

```
# Authentication & Session 开发协作契约

> 状态：`Ready for Development`（2026-09-14；设计 AI（项目负责人授权），ADR 0014）
> Owner：`A`；系统基线：`Baselined — 5d11769`；依赖波次：Wave 1<br>
> 唯一公开入口：`package:locatemy/features/authentication_session/authentication_session.dart`<br>
> 定义完成：消费者可仅凭本文件的公开 seam 区分真实会话、确认状态和失败，并安全完成登录、退出和换号协作。
```

本文件是 Authentication & Session 唯一的跨 Owner 开发协作契约，也是人类阅读 PDF 的 Main 源。它固定公开 Dart 声明、结果语义、数据边界和联合验收；`lib/features/authentication\_session/` 内的 Widget、状态管理、Supabase SDK 映射、重试、私有文件和测试组织由 Owner 决定。

0. 固定阅读顺序与四项 Readiness

实施者和审查者按以下顺序读取；后项只补充前项已固定的事实，不能改写其权威语义：

- 1. [产品认证事实](../knowledge_base/locatemy_product/features/authentication.md)与[账户事实](../knowledge_base/locatemy_product/features/account.md)；
- 2. [Feature map](../system/feature-map.md#authentication-session-interface 注册表)(../system/interfaces.md)、[FLOW-01](../system/flows.md#flow-01启动注册登录退出与账户切换)；
- 3. [Capability Traceability](../system/capability-traceability.md#data-ownership 数据所有权)(../system/data-ownership.md)、[Schema Catalog](../data/schema-catalog.md#身份与账户业务对象)和[风险登记](../system/risks-and-decisions.md#session-01-关闭证据)；
- 4. 本契约；需要生成或使用人类 PDF 工作包时，最后读 [ADR 0014](../adr/0014-versioned-pdf-development-documentation-packages.md) 与 [handoff 规范](../handoff/README.md)。

Readiness	本 Feature 的可核查证据	结论
---	---	---
责任与范围	`AUTH-01`、`AUTH-02`、`AUTH-03`、`ACCOUNT-07`	唯一归属 A；门控
契约与消费者	`AUTH-001`	的一个公开入口、声明、失败语义、调用顺序和 fake 场景均在第 3 节；消费者
交互	已就绪	
数据与安全	`auth.users` 和 `profiles`	的访问及权限只引用 Schema Catalog；无
seam	已就绪	
验收与风险	第 5 节覆盖 canonical `AT-*`；`RISK-SESSION-01`	已关闭，依赖版本

1. 任务成果、责任与依赖

- 用户可用邮箱密码登录当前设备；失败有可理解的修正或重试路径。
- 用户可注册；界面明确区分已建立会话、须查收验证邮件和未建立会话的失败；资料登记失败不否定认证结果。
- 账户页只依据 Auth 的真实邮箱与确认状态呈现；未知不显示“已验证”。

- 退出只结束当前设备会话；换号时先屏蔽和清理旧账户私有内容，不影响其他设备或远端业务记录。

Owner	负责	不负责	依赖
---	---	---	---
Authentication & Session (A)	邮箱密码认证、当前设备会话、真实邮箱 / 确认事实、可选注册资料清理、账户业务页面、其他设备会话 封装 Supabase Auth 和直接的 `profiles` 访问 Application Shell 认证门控、认证入口组合、退出 / 换号编排和私有 UI 屏障 Auth SDK 用 `PRIVACY-001.open` Account Privacy account scope 的 opened/closed 与本机清理 认证与导出 id; 仅 Shell 发起 Account Center 呈现真实邮箱 / 确认状态及退出意图 直读 Auth 或 `profiles` 注册		

包含登录、注册、真实邮箱确认、会话恢复 / 结束、可选注册资料及 `ACCOUNT-07` 的认证一侧。不包含 Magic Link 忘记密码、固定 Verified User 文案和 Account Privacy 的清理实现。产品事实见 [认证] (../knowledge_base/locatemy_product/features/authentication.md)、[账户] (../knowledge_base/locatemy_product/features/account.md)；系统流程见 [FLOW-01] (../system/flows.md#flow-01启动注册登录退出与账户切换)。

2. 需要调用的 Interface

Interface 卡: Supabase Auth seam (`AUTH-002`, 仅 A)

卡项	固定内容
---	---
提供者 / 消费者	Authentication & Session (A) / Authentication & Session (A)
Owner 暴露	SDK。
目的	将 Supabase Auth 的当前设备会话、真实邮箱和邮箱确认事实收敛为 `AUTH-001`。
输入 / 成功	邮箱密码动作或当前设备恢复；只有明确有效或成功恢复的会话才成为认证事实。
失败 / 恢复	过期缓存、刷新中、刷新失败、离线无法确认和远端拒绝均不打开私有 scope；映射为 `AUTH-001`。
权限 / 副作用	A 是唯一直接调用 Auth 与直接写 `profiles` 的 Owner；SDK 方法、token 与本地存储。

直接访问对象	用途与不可变规则
---	---
`auth.users`	提供 account id、真实 email、email confirmed state 与 email confirmed 时间，且不由 `profiles` 推断邮箱或确认状态。
`profiles`	仅认证后写可选 username/avatar/bio；`id` 等于当前 Auth account id；返回资料结果，不撤销或伪造认证结果。

字段、RLS 与迁移唯一权威为 [Schema Catalog] (../data/schema-catalog.md#身份与权限)。

3. 必须提供的 Interface

Interface 卡: 当前设备会话 (`AUTH-001`)

提供者: Authentication & Session (A)

 消费者: Application Shell、Account Privacy、Account Center

 唯一公开 import: `package:locatemy/features/authentication\_session.dart`

卡项	固定内容
目的	为消费者提供当前设备会话、不可伪造的 account id、真实邮箱及确认状态，并承载登录、注册和当前设备退出
输入	`restoreSession`、`watchSession`、邮箱密码登录、注册字段、退出意图；各调用的字段
成功结果	明确 authenticated 的账户、明确无会话、注册后的 authenticated 或 verified 设备会话。
失败与恢复	失败由第 3 节 enums 区分修正、重试、升级或停在门控；消费者不解析字符串，也不从旧 snapshot 记录或其他设备会话。
权限与副作用	本 Interface 不导航、不打开/关闭 privacy scope、不清理其他 Owner 数据
次序与账户隔离	Shell 以同一 account id 调用 `PRIVACY-001.open`，仅 `open`、`close` 顺序固定于“副作用、权限与调用次序”。

消费者只能 import 此入口，不得 import `lib/features/authentication\_session`。A 应先提交下列 **声明** 及最小公共类型；函数体由 A 亲自编写。

```

dart
abstract interface class AuthenticationSession {
  Future<SessionSnapshot> restoreSession();
  Stream<SessionSnapshot> watchSession();
  Future<SignInOutcome> signIn({
    required String email, required String password,
  });
  Future<RegistrationOutcome> register({
    required String email,
    required String password,
    required String passwordConfirmation,
    String? username,
  });
  Future<SignOutOutcome> signOut();
}

sealed class SessionSnapshot {
  const SessionSnapshot();
}

final class AuthenticatedSession extends SessionSnapshot {
  final AuthenticatedAccount account;
  const AuthenticatedSession(this.account);
}

final class UnauthenticatedSession extends SessionSnapshot {

```

```

    const UnauthenticatedSession();
}
final class SessionUnavailable extends SessionSnapshot {
    final SessionFailure failure;
    const SessionUnavailable(this.failure);
}

final class AuthenticatedAccount {
    final String accountId;
    final String email;
    final EmailConfirmation confirmation;
    const AuthenticatedAccount({
        required this.accountId,
        required this.email,
        required this.confirmation,
    });
}
enum EmailConfirmation { confirmed, verificationRequired, unavailable }

sealed class SignInOutcome {
    const SignInOutcome();
}
final class SignInSucceeded extends SignInOutcome {
    final AuthenticatedAccount account;
    const SignInSucceeded(this.account);
}
final class SignInRejected extends SignInOutcome {
    final SignInFailure failure;
    const SignInRejected(this.failure);
}

sealed class RegistrationOutcome {
    const RegistrationOutcome();
}
final class RegistrationAuthenticated extends RegistrationOutcome {
    final AuthenticatedAccount account;
    final ProfileRegistrationOutcome profile;
    const RegistrationAuthenticated(this.account, this.profile);
}
final class RegistrationVerificationRequired extends RegistrationOutcome {
    final String email;
    final ProfileRegistrationOutcome profile;
    const RegistrationVerificationRequired(this.email, this.profile);
}

```

```

}
final class RegistrationRejected extends RegistrationOutcome {
    final RegistrationFailure failure;
    const RegistrationRejected(this.failure);
}

sealed class ProfileRegistrationOutcome {
    const ProfileRegistrationOutcome();
}
final class ProfileRegistered extends ProfileRegistrationOutcome {
    const ProfileRegistered();
}
final class ProfileRegistrationSkipped extends ProfileRegistrationOutcome {
    const ProfileRegistrationSkipped();
}
final class ProfileRegistrationFailed extends ProfileRegistrationOutcome {
    final ProfileFailure failure;
    const ProfileRegistrationFailed(this.failure);
}

sealed class SignOutOutcome {
    const SignOutOutcome();
}
final class SignOutSucceeded extends SignOutOutcome {
    const SignOutSucceeded();
}
final class SignOutRejected extends SignOutOutcome {
    final SignOutFailure failure;
    const SignOutRejected(this.failure);
}

enum SessionFailure { retryableUnavailable, unsupportedClient, remoteRejected }
enum SignInFailure { invalidInput, invalidCredentials, retryableUnavailable, unsupportedClient }
enum RegistrationFailure { invalidInput, accountAlreadyExists, retryableUnavailable, unsupportedClient }
enum ProfileFailure { retryableUnavailable, permissionDenied, remoteRejected }
enum SignOutFailure { retryableUnavailable, remoteRejected, unsupportedClient }

```

这规定协作形状，不规定 `sealed-class` 的内部文件组织、`SDK-enum` 或错误文案。新增公共成员 / 结果变体须按第 6 节。

输入

调用	输入与约束	调用者责任
<code>`restoreSession`</code>	/ <code>`watchSession`</code>	无 <code>Shell</code> 只在认证门控未打开私有路由时调用。
<code>`signIn`</code>	非空 <code>`email`</code> 、 <code>`password`</code>	密码仅本次表单 / 调用存在；不得记录、持久化或传给其他模块。
<code>`register`</code>	非空 <code>`email`</code> 、 <code>`password`</code> 、 <code>`passwordConfirmation`</code> ；可空 <code>`username`</code>	必须等于 <code>password</code> 才提交；无 <code>username</code> 表示跳过资料。密码强度与文案由 A 在产品事实范围内决定。
<code>`signOut`</code>	无	仅 <code>Shell</code> 在旧私有 UI 已屏蔽后调用； <code>Account Center</code> 不直接调用。

输出、失败与处理

结果	意义	消费者必须处理
<code>`AuthenticatedSession`</code> 、 <code>`SignInSucceeded`</code> 、 <code>`RegistrationAuthenticated`</code>	身份和真实邮箱事实	<code>Shell</code> 以同一 <code>accountId</code> 调 <code>`PRIVACY-001`</code> ， <code>Account Center</code> 使用 <code>confirmation</code> 。
<code>`UnauthenticatedSession`</code>	明确没有可用会话	显示认证入口，不显示私有 UI。
<code>`SessionUnavailable`</code>	无法安全确认会话	<code>failure</code> 区分可重试网络、客户端不支持、远端拒绝 UI。
<code>`RegistrationVerificationRequired`</code>	注册须查收验证邮件	<code>email</code> 是注册目标邮箱，仍以 <code>SessionSnapshot</code> 开 <code>scope</code> 。
<code>`confirmed`</code> / <code>`verificationRequired`</code> / <code>`unavailable`</code>	Auth 验证、待验证、状态不可得；未知不可猜测。	
任一 <code>`*Rejected`</code>	本次动作没有对应成功语义	使用 <code>enum</code> 区分修正、重试或升级；不解析错误字符串，只返回 <code>enum</code> 。
<code>`ProfileRegistrationFailed`</code>	认证 / 验证结果仍有效，只有资料未登记	显示资料可重试，结束 <code>scope</code> 。

``watchSession`` 的首个和后续 `snapshot` 语义一致；过去 `authenticated` 值不能授权继续显示。

副作用、权限与调用次序

- ``accountId`` 唯一来自 `Auth`；公开类型不暴露密码、`token`、`SDK-payload` 或 ``profile``。
- 登录 / 注册 / 恢复只报告认证事实，不导航、不打开 `scope`、不清理别的模块数据。
- ``signOut`` 只结束 `**当前设备**` 会话；不会删除远端业务记录或结束其他设备会话。
- `Shell` 的退出 / 换号固定顺序：屏蔽旧私有 UI 和新业务意图 → ``signOut`` → ``PRIVACY-001`` (旧 `AccountId``) → 两者成功后才显示普通登录入口。任一步失败仍无私有内容且可恢复。
- 已打开 `scope` 的 `accountId` 与 ``AuthenticatedSession`` 不一致时，`Shell` 不显示私有内容；按 `FLOW-01` 关闭旧范围后重新认证。

最小调用示例 (Shell)

```
` `` ` dart
final snapshot = await authenticationSession.restoreSession();
switch (snapshot) {
  case AuthenticatedSession(:final account):
    // 以 account.accountId 调用 PRIVACY-001.open;仅 opened 后进入主应用
  case UnauthenticatedSession():
    // 显示登录 / 注册入口。
  case SessionUnavailable(:final failure):
    // 保持门控, 按 failure 提供恢复路径。
}
` `` `
```

这是 seam 用法说明, 不是可提交的 Shell 实现。

Fake Adapter 场景

Shell 用 fake `AuthenticationSession` 分别返回 `AuthenticatedSession`、`UnauthenticatedSession`、`SessionUnavailable(retryable)`、验证仅在 A 的 `PRIVACY-001.open` 返回 opened 后建立私有 UI。Account Center 用同一 fake 的三种 `EmailConfirmation`, 验证 unavailable 不会被呈现为 verified。无需 Supabase 或 A 的生产实现。

4. 协作交付顺序

- 1. **A: 先合入公开 seam。 ** 创建唯一入口、上述声明及最小 fake; 合入前消费者不依赖 A 内部文件。
- 2. **A: 完成真实 Adapter 与表单。 ** 映射 Supabase Auth/`profiles` 到这些结果, 并编写真实 Adapter 契约测试。
- 3. **消费者: 并行使用 fake。 ** Shell 完成门控和 FLOW-01; Account Privacy 只接收 accountId; Account Center 只用 `AuthenticatedAccount`。
- 4. **共同联调。 ** 接入生产 seam, 仅为第 5 节保留必要跨模块流测试。

5. 联合验收

场景	参与 Owner	操作	可观察结果	追踪
冷启动恢复会话	A、Shell、Privacy		无会话、有效 A、不可确认会话	分别为认证入口、仅 scope、`AUTH-01`、`AT-AUTH-01`
登录凭据或网络失败	A、Shell	非法 / 错误凭据、网络不可用、客户端不支持	可区分失败和恢复路径; 无 scope、`AUTH-01`、`AT-OUT-02`	
注册与真实确认	A、Shell、Account Center		authenticated、verification 状态分明; 仅 Auth 确认事实可显示 verified	`AUTH-02`、`AUTH-03`、`AT-AUTH-02`
当前设备退出	A、Shell、Privacy	确认退出; 注入 Auth / 清理失败	成功到普通登录; 失败仍无 scope、`AUTH-07`、`AT-OUT-01`、`AT-OUT-02`	
从 A 换到 B	A、Shell、Privacy、私有 Owner	A 已打开后退出并登录 B	B scope、数据	`ACCOUNT-07`、`AT-SWITCH-01`

6. 实现自由、阻塞项与参考

Owner 可决定 `src/` 内的拆分、Widget、状态管理、SDK Adapter、刷新 / 重试 / 取消、局部校验和测试。任何变更都必须暂停协商：变更唯一公开入口、公共声明 / 结果 / 次序 / 权限；需变更 `profiles` schema / RLS；或 Superset 版本使 `RISK-SESSION-01` 的锁定结论失效。

当前阻塞项：无。依赖版本变化时重开 `RISK-SESSION-01`。权威参考：[Schema Catalog](../catalog/schema-catalog.md#身份与账户业务对象)、[Interface 注册表](../system/interface.md#会话风险)(../system/risks-and-decisions.md#risk-session-01-关闭证据)。

7. 契约变更与完成检查

公共 Interface 变更须由提供方说明原因和受影响消费者，所有受影响消费者确认；同一 PR 更新公开声明、本契约、受影响 Adapter 测试和 PDF。Git / PR 保存历史；不使用文档版本、checksum、Manifest、Generate 或 Development Release。

- [x] 四项 Readiness 在第 0 节均有可核查证据。
- [x] `AUTH-001` 和 `AUTH-002` 均有完整 Interface 卡；`AUTH-001` 只有一个 retry point。
- [x] Dart 仅表达跨 Owner 的公开声明；不含方法实现、SDK 调用、SQL、测试实现或 Widget 结构。
- [x] fake 场景和联合验收覆盖恢复 / 门控、注册 / 验证、退出 / 清理和 A→B 换号。